
THE CHRONICLES OF RAG: THE RETRIEVER, THE CHUNK AND THE GENERATOR

PREPRINT

*Paulo Finardi

*Leonardo Avila
Marcos Piau

Rodrigo Castaldoni
Pablo Costa

Pedro Gengo
Vinicius Caridá

Celio Larcher

22h, Brazil

email: {pfinardi, leonardo.bernardi.avila, castaldoniro, pedro.gengo.lourenco, celiolarcher, marcos.piau.vieira, pablo.botton.costa, vfcarida}@gmail.com

* Both authors contributed equally to this research.

ABSTRACT

Retrieval Augmented Generation (RAG) has become one of the most popular paradigms for enabling LLMs to access external data, and also as a mechanism for grounding to mitigate against hallucinations. When implementing RAG you can face several challenges like effective integration of retrieval models, efficient representation learning, data diversity, computational efficiency optimization, evaluation, and quality of text generation. Given all these challenges, every day a new technique to improve RAG appears, making it unfeasible to experiment with all combinations for your problem. In this context, this paper presents good practices to implement, optimize, and evaluate RAG for the Brazilian Portuguese language, focusing on the establishment of a simple pipeline for inference and experiments. We explored a diverse set of methods to answer questions about the first Harry Potter book. To generate the answers we used the OpenAI's gpt-4, gpt-4-1106-preview, gpt-3.5-turbo-1106, and Google's Gemini Pro. Focusing on the quality of the retriever, our approach achieved an improvement of MRR@10 by 35.4% compared to the baseline. When optimizing the input size in the application, we observed that it is possible to further enhance it by 2.4%. Finally, we present the complete architecture of the RAG with our recommendations. As result, we moved from a baseline of 57.88% to a maximum relative score of 98.61%.

1 Introduction

The rise of Large Language Models (LLMs) has changed the way we approach Artificial Intelligence (AI) applications. Their ability to answer different user queries in different domains allow these models to show a notable performance in a wide range of tasks like translation, summarizing, question answering, and many others [1]. However, there are a lot of open challenges when it comes to problems that require answers based on updated information, and external data, that were not available in the training data.

In order to overcome this challenge, a technique called Retrieval Augmented Generation (RAG) [2] was developed. This approach aims to solve the limitation of the need for external data, by fetching and incorporating this information in the prompt. With this, the model can generate more cohesive answers about subjects and data not seen during the training, decreasing the occurrence of hallucinations [3]. Nevertheless, this approach adds a new layer of challenges since it requires the development of a trustworthy retriever pipeline, given that the quality of the final answer can be highly affected if the retrieved text is not relevant to the user query [4].

The landscape of RAG is rapidly expanding, with a constant influx of new papers introducing diverse implementations [5]. Each of these variants proposes technical modifications or enhancements, such as different retrieval mechanisms, augmentation techniques, or fine-tuning methodologies. This proliferation, while a testament to the field's dynamism,

presents a substantial challenge for AI practitioners. The task of methodically experimenting with, and critically evaluating, each variant’s performance, scalability, and applicability becomes increasingly complex.

In this paper, we present a comprehensive series of experiments focused on the application of RAG specifically tailored for Brazilian Portuguese. Our research delves into evaluating various retrieval techniques, including both sparse and dense retrievers. Additionally, we explore two chunking strategies (naive and sentence window) to optimize the integration of retrieved information into the generation process. We also investigate the impact of the positioning of documents within the prompt, analyzing how this influences the overall quality and relevance of the generated content. Finally, our experiments extend to comparing the performance of different LLMs, notably GPT-4 and Gemini, in their ability to effectively incorporate the retrieved information and produce coherent, contextually accurate responses. This paper aims to provide valuable insights and practical guidelines for implementing RAG in Brazilian Portuguese.

Our main contributions are summarized as follows: 1) we propose a methodology to prepare a dataset in a format that allows quantifying the quality of the different steps in an RAG system. 2) We proposed a metric (maximum relative score) that allow us to direct quantify the existent gap between each approach and a perfect RAG system. 3) We discuss and compare different implementations, showing good practices and optimizations that can be used when developing a RAG system.

2 Data Preparation

The chosen dataset was the first Harry Potter book in its Brazilian Portuguese version. This choice is motivated since it is a well known book, and both the Gemini Pro and OpenAI models can answer general questions on the subject. Additionally, over the application of the standard ChatGPT tokenizer `cl100k_base`, we observed that there are approximately 140,000 tokens in total, allowing the creation of prompts containing the entire book. Following, a dataset consisting of questions and corresponding answers was developed, with both question and answer generated by the *gpt-4* model and based on a reference chunk.

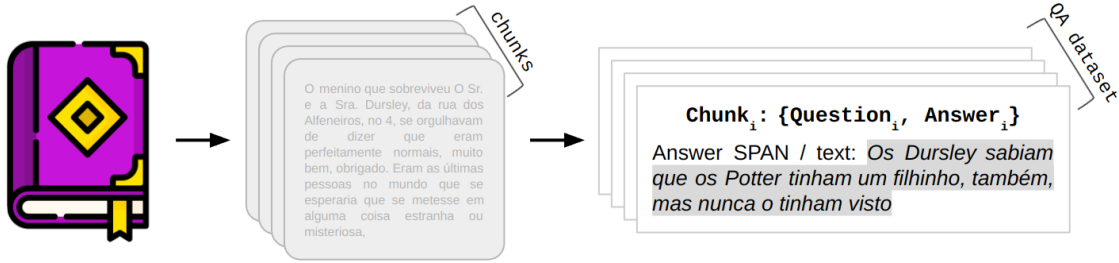


Figure 1: From a large document (book), chunks were created, and for each chunk, a question and an answer were generated using *gpt-4*, where the answer is contained within the chunk.

Figure 1 shows the data preparation process. Initially, the dataset was break into chunks with 1000 tokens each, without overlapping, resulting in 140 chunks. Then, using the prompt described in Appendix A, a $\{question, answer\}$ pair was created for each chunk in the style of the SQuAD dataset [6], meaning that the answer to the question is present within the reference text (chunk).

3 How to Evaluate

The contextual comparison of two text samples is not a straightforward task. For instance, despite the sentence₁ = "Brazil has won 5 FIFA World Cup titles.", and sentence₂ = "Brazil is the five-time champion of the FIFA World Cup." (both sentences translated into English for convenience) convey the same meaning, traditional metrics such as BLEU [7] and ROUGE [8] score may not be able to capture such similarity. Specifically, for the example cited:

- BLEU score [sentence₁, sentence₂] = 0.33
- ROUGE score [sentence₁, sentence₂] = 0.22

Therefore, an approach widely used in the literature is to employ *gpt-4* to provide a score based on a given prompt, a concept similar to what was done in the G-Eval work [9]. In this work, a scoring system divided into 5 categories to compare two texts was devised, with scores defined as following (translated into English for convenience):

- score 1: The answer has no relation to the reference.
- score 3: The answer has little relevance but is not aligned with the reference.
- score 5: The answer has moderate relevance but contains inaccuracies.
- score 7: The answer is aligned with the reference but has minor omissions.
- score 10: The answer is completely accurate and aligns perfectly with the reference.

The prompt used in the evaluation is shown in Appendix A. Our approach uses a one-shot technique for each scoring category. Although, we believe that the evaluation could become more robust and deterministic with the addition of more few-shot examples for each scoring category, these possible variations were not explored in this work.

3.1 Relative Maximum Score

In order to assess performance variation for the followings experiments, we created a metric called the relative maximum score, which corresponds to the score given by a model when evaluating the correct combination of question and chunk for all pairs of a given dataset. Through this approach, it is possible to obtain the maximum score that an evaluated LLM could reach for a RAG system.

The Table 1 presents results for the custom dataset created in Section 2, by using different LLMs to generate the answer and the gpt-4 score system previously defined.

Table 1: Relative maximum score in 140 questions from the created QA Harry Potter dataset.

Model	Relative Maximum
gpt-4	7.55
gpt-4-1106-preview	7.32
gpt-3.5-turbo-1106	7.35
Gemini Pro	7.52

Despite configuring all the seeds and reusing the prompts, the relative maximum in our data was approximately 7.4 average points. This points out that, even with a perfect retriever strategy, the RAG system is not able to achieve a perfect score in this dataset by using these LLM models.

For now on, all experiments in this study were assessed in terms of both the relative maximum and the percentage degradation with respect to the relative maximum, as defined in Equation 1.

$$\text{degradation score} = 1 - \frac{\text{experiment score}}{\text{relative maximum}} \quad (1)$$

With this score, we are able to address the problems regarding the retriever system itself, instead of having a vague idea of where is the main gap of our pipeline.

4 Introductory Experiments

In this section, we will establish a baseline for the metrics defined in Section 3. Additionally, we will apply techniques of lower complexity and compare the results with the baseline. It's worth noting that we did not explore prompt engineering techniques, although we are aware that prompt engineering has a direct impact on performance, as demonstrated in [10].

4.1 Baseline: no context

We are aware that LLMs are trained on a massive dataset that covers virtually the entire web. This circumstance, coupled with the popularity of the Harry Potter universe, forms a robust hypothesis for testing questions in isolation on OpenAI models. The answer for basic questions such as "Who is Harry Potter?", "Who killed Dumbledore?", and "What are Harry Potter's main friends?", ChatGPT were correctly answered with precision. However, we observed that when dealing with more detailed questions, the performance was only reasonable. Below are two examples of detailed questions (translated into English for convenience):

- **Query:** What was Harry’s strategy to stop Quirrell from focusing on the mirror and what did he try to do to discover the location of the Philosopher’s Stone?
 - **Answer:** What occurred to Harry was to keep Quirrell talking to stop him from focusing on the mirror.
- **Query:** What model of broom did Harry Potter receive and who mentioned the special circumstances to Professor Flitwick?
 - **Answer:** The model of the broom that Harry Potter received is a Nimbus 2000 and it was Professor. Minerva who mentioned the special circumstances to Professor Flitwick.

The Table 2 shows the baseline results obtained using some known LLMs for the 140 questions built as described in section 2 and evaluated as described in section 3. For this task, no retrieved context were used, only the question.

Table 2: Performance of the External Knowledge experiment.

Model	Average Score	Degradation
gpt-4	5.35	-29.1%
gpt-4-1106-preview	5.06	-30.9%
gpt-3.5-turbo-1106	4.91	-32.8%
Gemini Pro	3.81	-50.8%

4.2 Long Context

In comparison to the GPT 1 and 2 models [11, 12], which handle up to 1024 input tokens, the gpt-4-1106-preview model stands out for its remarkable ability to process up to 128k input tokens. This represents an approximately 128 times increase in input capacity over just four years in model development.

The specific architecture of gpt-4 has not been disclosed, but it is believed that this model has not been pre-trained with a 128k token input context [13]. Perhaps a post-pre-training technique could have been used, which would have made it possible to expand the number of input tokens [14, 15]. However, it is essential to note that such a technique may show degradation as the expansion limit is reached [16]. Similar to Recurrent Neural Networks (RNNs), which theoretically have an infinite context, disregarding performance limitations and vanish-gradients, we are interested in evaluating the performance of gpt-4-1106-preview over its 128k tokens.

To assess the impact of the gpt-4-1106-preview full context capacity on the model’s response, we proceed with a similar analysis of "Lost in The Middle" [17] in our dataset. This analysis explores the model output for a given question while changing the position of the answer throughout the prompt. To conduct this experiment, the depth of the chunk containing the answer for the question was altered in increments of 10% of the total number of tokens in the context’s prompt. Thus, on the y-axis, there are 11 variations of answer depth, represented by 0%, 10%, 20%, . . . , 90%, 100%, and the x-axis represents the quantity of tokens used as input in the context, as shown in Figure 2. The colors represents the experiment score, where the greener the better.

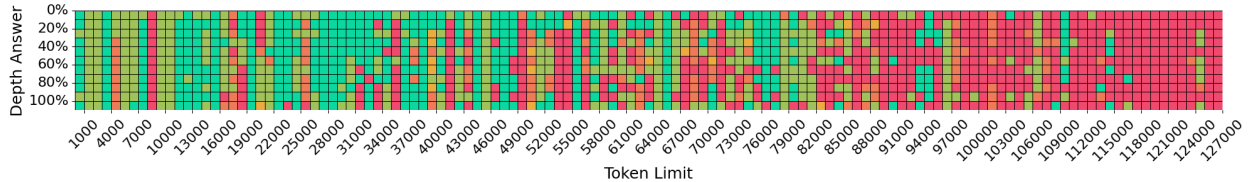


Figure 2: Performance of gpt-4-1106-preview on the Harry Potter dataset, x-axis: spaced at every 1,000 tokens of input from the document, y-axis: represents the depth at which the answer is located in the document. The greener the better. Image based on Gregory repository [18].

For instance, for $(x = 100,000, y = 40\%)$, there are (39 chunks, followed by the chunk containing the answer, then by the remaining 60 chunks, making up the 100,000 tokens in the input context. Based on Figure 2, we can also see when increasing the input length, we see a strong degradation in the score. Besides that, the Figure 3 shows that answers located in the interval of (40%, 80%) exhibit the worst performance, as documented in the article "Lost In The Middle" [17].

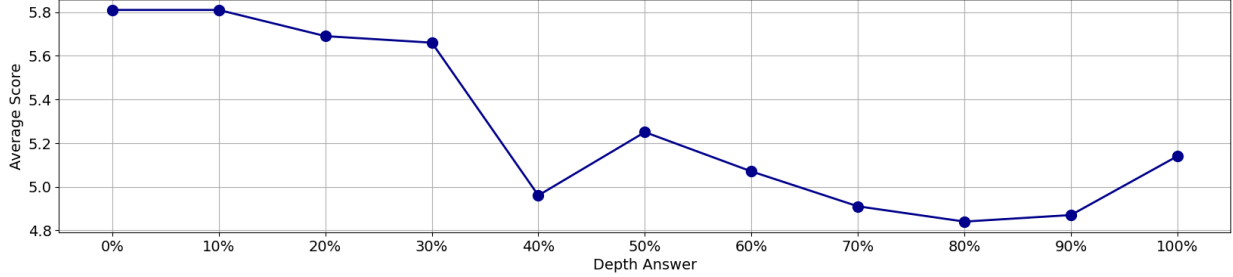


Figure 3: Average performance analysis of gpt-4-1006-preview using 128k tokens context per answer depth.

4.3 RAG Naive

Initially, a straightforward approach for RAG will be done using the llama-index [19], employing all default hyper-parameters and using chunk retrieval by cosine similarity with the ADA-002 embedding. Figure 4 depicts the basic diagram of how the problem is addressed.

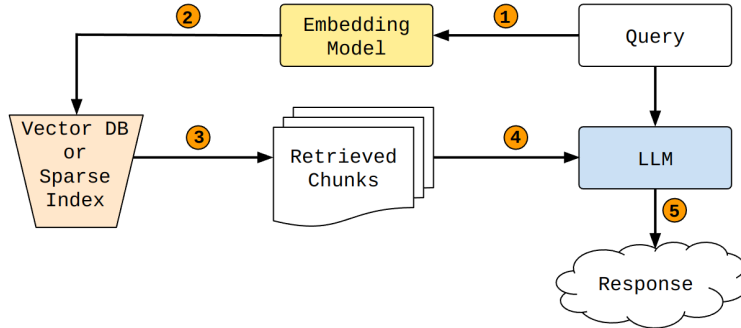


Figure 4: 1. Pass the query to the embedding model to represent its semantics as an embedded query vector; 2. Transfer the embedded query vector to vector database or sparse index (BM25); 3. Fetch the top-k relevant chunks, determined by retriever algorithm; 4. Forward the query text and the chunks retrieved to Large Language Model (LLM); 5. Use the LLM to produce a response based on the prompt filled by the retrieved content.

The Table 3 shows the average and degradation metrics for this approach using 2 retrieved chunks.

Table 3: Performance of the RAG naive.

Model	Average Score	Degradation
gpt-4	6.04	-20%
gpt-4-1106-preview	5.74	-21.6%
gpt-3.5-turbo-1106	5.80	-21.0%

5 Advanced Experiments

The studies and experiments outlined in section 4 have shown unsatisfactory performance, marked by a degradation of at least 20% compared to the peak relative performance. Therefore, in this section, we explore various retrieval approaches for the RAG, recognizing that the quality of the retriever is a crucial factor in enhancing performance for this type of problem. We conducted an evaluation covering both sparse and dense search, a hybrid method, and even a multi-stage architecture using a reranker.

In pursuit of code debugging flexibility and easier customization at each stage, we chose not to utilize an RAG framework (like LangChain or Llama-Index). For a comprehensive guide on debugging RAG and more details about retrieval systems, refer to [20] and [21].

5.1 Retrievers

When deploying retrieval systems, it is essential to achieve a balance between “effectiveness” (*How good are the results returned?*) and “efficiency” (*How much time it takes to return the results?* or *How much resources are used in terms of disk/RAM/GPU?*). This balance ensures that latency, result quality, and computational budget remain within our application’s required limits. This work will exclusively focus on effectiveness measures to quantify the retrievers methods quality.

In our retriever experiments, the evaluation strategy centers around assessing how well the retriever performs in retrieving relevant information based on each given query q_i . To achieve this, we employ the concept of recall. It is defined as the fraction of the relevant documents for a given query q_i that are successfully retrieved in a ranked list R [21]. This metric is based on binary relevance judgments, assuming that documents are either relevant or not [21]. In this paper, each chunk is considered a document and only the respective chunk d_i is considered relevant to the query q_i . While recall is easy to interpret, it does not consider the specific rank positions in which the relevant chunk appears in R .

To overcome this limitation, we introduce Reciprocal Rank (RR) into our analysis. In this metric, the rank of the first relevant document to the query in R is used to compute the RR score [21]. Therefore, Reciprocal Rank offers a more nuanced evaluation by assigning a higher value when the relevant chunk is returned in the early positions of our retrievers given the respective query.

Recall and Reciprocal Rank were evaluated at a specific cutoff so the measures are presented as $R@k$ and $MRR@k$. For each query, its results are evaluated and their mean serves as an aggregated measure of effectiveness of a given retriever method. The retrievers are introduced below.

In the category of sparse retrievers, we emphasize the BM25, a technique grounded in statistical weighting to assess relevance between search terms and documents. BM25 employs a scoring function that takes into account term frequency and document length, offering an efficient approach for retrieving pertinent information and is typically used as a strong baseline. However, it is exact-match based and can be powerless when query and document are relevant to each other but has no common words.

On the other hand, when exploring dense retrievers, we often encounter approaches based on the called bi-encoder design [22]. The bi-encoder independently encodes queries and documents, creating separate vector representations before calculating similarity. An advantage of this approach is that it can be initialized ‘offline’: document embeddings can be precomputed, leaving only the query embedding being calculated at search time, reducing latency.

The hybrid search technique aims to leverage the best of both sparse and dense search approaches. Given a question, both searches are conducted in parallel, generating two lists of candidate documents to answer it. The challenge then lies in combining the two results in the best possible way, ensuring that the final hybrid list surpasses the individual searches. Essentially, we can conceptualize it as a voting system, where each searcher casts a vote on the relevance of a document to a given query, and in the end, the opinions are combined to produce a better result.

The multi-stage search architecture is based on the retrieve-and-rerank pipeline. In the first stage, a retriever with good recall is typically used to perform an initial filtering of the documents to be returned. From this narrowed-down list, these candidate documents are then sent to a second stage, which involves higher computational complexity, to rerank them and enhance the final effectiveness of the system.

Next, we provide more details about each retriever used.

5.1.1 BM25

Due to the user-friendly nature of BM25, its inclusion as a retriever method is always a welcome addition in RAG evaluations. A study that aligns with the same reasoning, albeit for a different application, can be found in [23], which illustrates the benefits of employing this algorithm to establish a robust baseline. Given that our data shares similarities with the SQuAD dataset, it is expected that many words from the query would be present in the chunk, contributing to the favorable effectiveness of BM25.

In the BM25 ranking function, k_1 and b are parameters shaping term saturation and document length normalization, respectively. The BM25 formula integrates these parameters to score the relevance of a document to a query, offering flexibility in adjusting k_1 and b for improving effectiveness in different retrieval scenarios.

Table 4: Comparison between BM25 packages using $k_1=0.82$ and $b=0.68$.

Recall@k	rank-bm25	Pyserini BM25	Pyserini Gain (%)
3	0.735	0.914	24.3
5	0.814	0.971	19.2
7	0.857	0.985	14.9
9	0.878	0.985	12.1

Effectiveness is also influenced by chosen BM25 implementation. Pyserini’s BM25 implementation incorporates an analyzer with preprocessing steps such as stemming and language-specific stop word removal. For the sake of comparison, we included results obtained using rank-bm25 [24], a basic implementation without preprocessing that is widely used in Python and integrated into libraries like LangChain and Llama-index. The results can be seen in Table 4.

In this work, the Pyserini BM25 implementation [25] was used in all experiments considering $k_1=0.82$ and $b=0.68$.

5.1.2 ADA-002

OpenAI does not disclose extensive details about the ADA-002 architecture; however, we employed this model in retrieval as the presented bi-encoder design (Figure 5): vector representations were constructed for all available chunks, and for each input query, its embedding was computed at search time. Subsequently, the similarity between the question and chunk were assessed using cosine similarity.

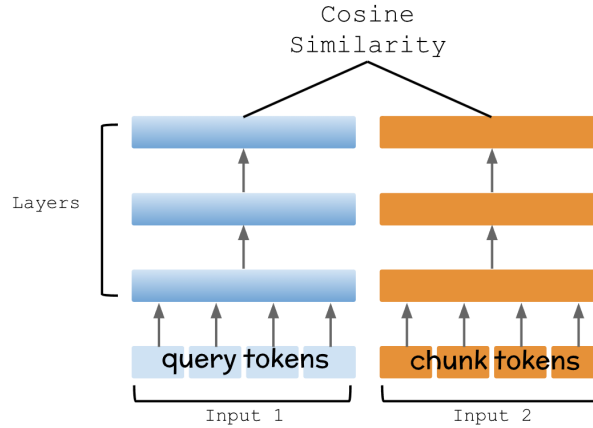


Figure 5: Bi-Encoder Architecture

Since we have no further details about ADA-002 we will refer to this approach only as dense retriever.

5.1.3 Custom ADA-002

The Custom ADA-002 approach was also utilized in the dense retriever configuration presented in Section 5.1.2. However, embedding customization played a key role in our attempt to enhance the overall representation.

Embedding customization is not limited solely to OpenAI’s embeddings; it is a technique applicable to other embeddings of the same kind. There is a significant variety of approaches to optimize a matrix, with one of them being

the application of Multiple Negative Ranking Loss, as presented in the article "Efficient Natural Language Response Suggestion for Smart Reply," Section 4.4 [26]. However, given our current focus on simplicity, we will reserve the exploration of this technique for future work. At this moment, we choose to utilize the Mean Squared Error (MSE) Loss.

For the fine-tuning stage, it is necessary to have two types of samples:

- positives: $[\text{question}_i, \text{chunk}_i, \text{label} = 1]$
- negatives: $[\text{question}_i, \text{chunk}_j, \text{label} = -1]$, for $i \neq j$

Often, as is the case with our dataset, only positive examples are available. However, through a simple and random shuffling, it is possible to generate negative examples. Demonstrating confidence in transfer learning, we found that a few examples were sufficient. Our final dataset consisted of approximately 400 examples, maintaining a 1 : 3 ratio between positive and negative examples.

The hyperparameters that exert the most significant impact on performance include the learning rate, batch size, and the number of dimensions in the projection matrix. The ADA-002 model has 1536 dimensions, and the projection matrix is of size $1536 \times N$, where $N \in \{1024, 2048, 4096\}$. In our experiments, we observed that 2048 dimensions resulted in the best accuracy.

This type of fine-tuning requires low GPU resources, with a training time of approximately 5 minutes using the A100 GPU. The model itself is straightforward, consisting of a matrix with dropout (to mitigate overfitting), followed by the hyperbolic tangent activation function, which provided additional accuracy gains in the training set.

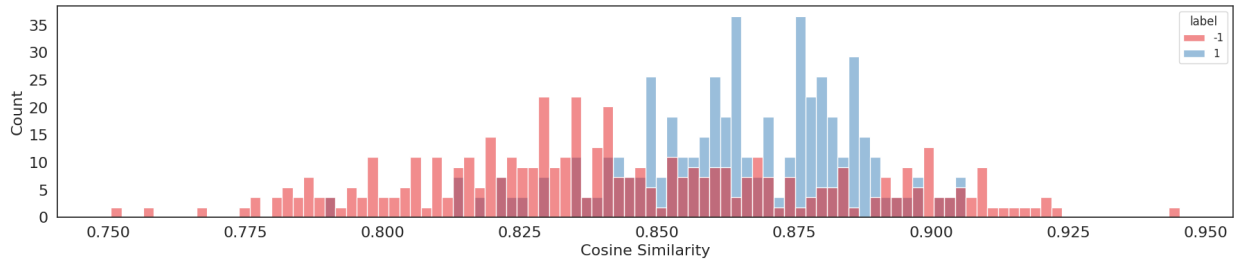


Figure 6: Cosine similarity of positive and negative classes in the ADA-002 embedding; note the significant overlap between the classes. Test accuracy (*before training*): 69.5%

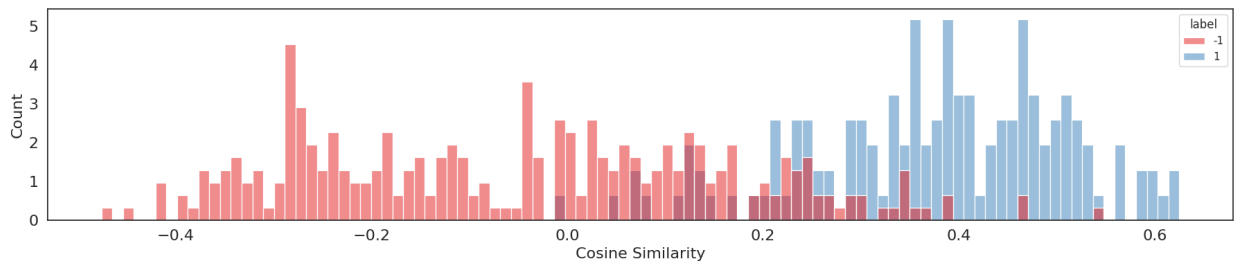


Figure 7: Cosine similarity of positive and negative classes in the customized embedding; the intersection between the classes is minimal. Test accuracy (*after training*): 84.3%

When analyzing the cosine similarity between positive and negative classes, we can observe the "shadow" shared by the histograms. In an ideal scenario, we desire the classes to be disjoint to ensure a clear definition of space. Figure 6 illustrates a significant shadow in the embedding before training, while Figure 7 shows the result after training. Both graphs are derived from the test set. Test accuracy also improved, leading to a better dense representation.

5.1.4 Hybrid Search

As stated before, hybrid search is applied when is necessary to combine results from two or more retrieval methods. A widely used algorithm to address this type of problem is known as Reciprocal Rank Fusion (RRF). For a document

set D and search results from different methods r in R , for each d in D , we can calculate the $RRF_{score}(d \in D)$ as follows [27]:

$$RRF_{score}(d \in D) = \sum_{r \in R} \frac{1}{k + r(d)}, \quad (2)$$

Considering that $1/r(d)$ is known as the reciprocal rank, where $r(d)$ represents the position at which the document d was retrieved by the search mechanism r . The term k is introduced to assist in controlling outlier systems [27].

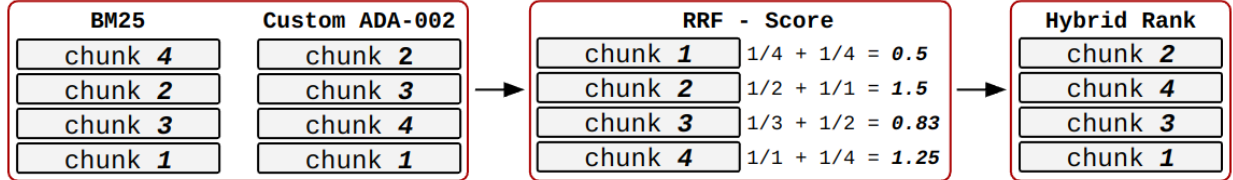


Figure 8: Hybrid Search schema with $k=1$.

Figure 8 shows how to calculate RRF_{score} for $k = 1$. In the example, we have four chunks that were retrieved in different orders by two search methods, BM25 (sparse search), and Custom ADA-002 (dense search). The reciprocal rank score is calculated for each chunk. These values are then summed, creating a new score. The final hybrid list is an ordering of chunks that uses this new score.

Table 5: Retriever comparison. Where MRR is the Mean Reciprocal Rank metric and R@k is the Recall.

Metric	Hybrid-BM25-ADA-002	Hybrid-BM25-Custom ADA-002
MRR@10	0.758	0.850
R@3	0.829	0.921
R@5	0.879	0.943
R@7	0.921	0.964
R@9	0.957	0.979

In our experiments, only Pyserini’s BM25 was tested as the sparse retriever, while ADA-002 and Custom ADA-002 were tested as dense retrievers. The hybrid combination that yielded the best results was the one that used BM25 and Custom ADA-002.

5.1.5 Reranker

The fundamental idea underlying multi-stage ranking is to divide document ranking into a sequence of stages. After an initial retrieval, which usually involves a sparse retriever or dense retriever, each subsequent stage re-evaluates and reranks the set of candidates forwarded from the preceding stage. Figure 9 represents a multi-stage pipeline where Pyserini BM25 performs the first stage and the candidate chunks are then re-evaluated by the reranker. After that, the final reranked list called retrieved chunks is presented as final result formed by k chunks.

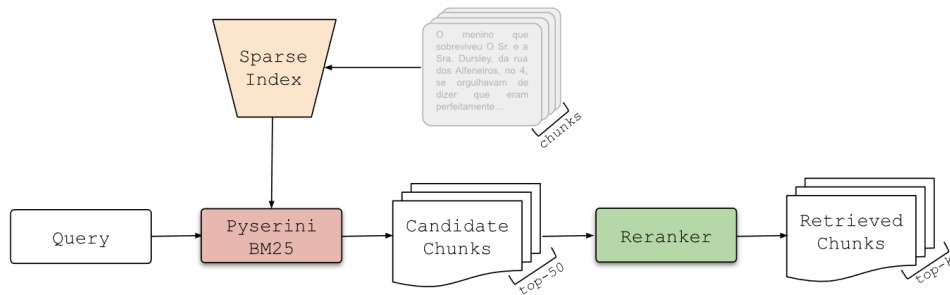


Figure 9: Reranker Pipeline

Transformers-based models are commonly employed as rerankers, leveraging their capability to enhance the effectiveness of information retrieval systems by capturing intricate relationships and contextual information within documents and queries.

The initial utilization of transformers within a multi-stage ranking framework was presented in the study [28]. Their proposed model, known as monoBERT, transforms the ranking process into a relevance classification problem. It achieves this by sorting texts based on the conditional probability $P(\text{Relevant} = 1 | d_i; q)$, where q is the query and d_i represents documents [21]. The model simultaneously processes both queries and documents. This simultaneous processing leads to a more enriched interaction between them, often resulting in improved effectiveness [29], [30]. However, these neural models have a substantial number of parameters and the scoring of query-document pairs occurs at inference time. This, in turn, increases computational costs and latency. This kind of model is also known by cross-encoder. Refer to Figure 10 for an illustration of the query-document pair processing.

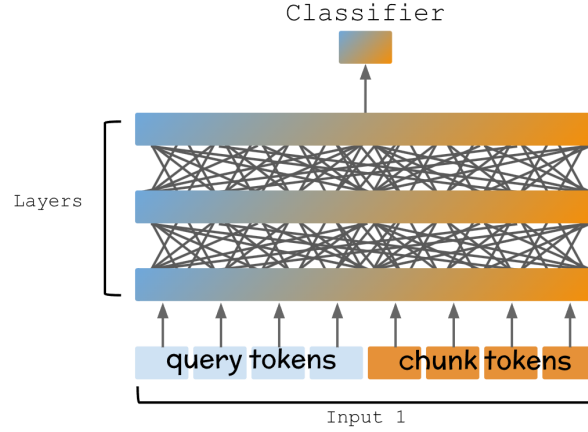


Figure 10: Cross-Encoder

In the other hand, monoT5 is a sequence-to-sequence reranker [31] that uses T5 models [32] to generate relevance scores between a pair of query and document. T5 models treat all tasks as *text-to-text*, requiring some adaptations for the query/document similarity task. During training, the format 'Query: {query} Document: {document} Relevant: ' is used, with labels yes if the document is relevant to the query, and no otherwise. At inference time, the same format as the training data is used to format pairs of queries and documents to feed the model, and a single-token greedy decode is performed. The score is then obtained by calculating the softmax value considering only the tokens no and yes, and selecting the value corresponding to the yes token. Note that the tokens no and yes are the tokens used in the version provided by [33], but the publication that introduced the monoT5 architecture, [31], uses the tokens false and true. Figure 11 contains an illustration of the monoT5 architecture's inference process.

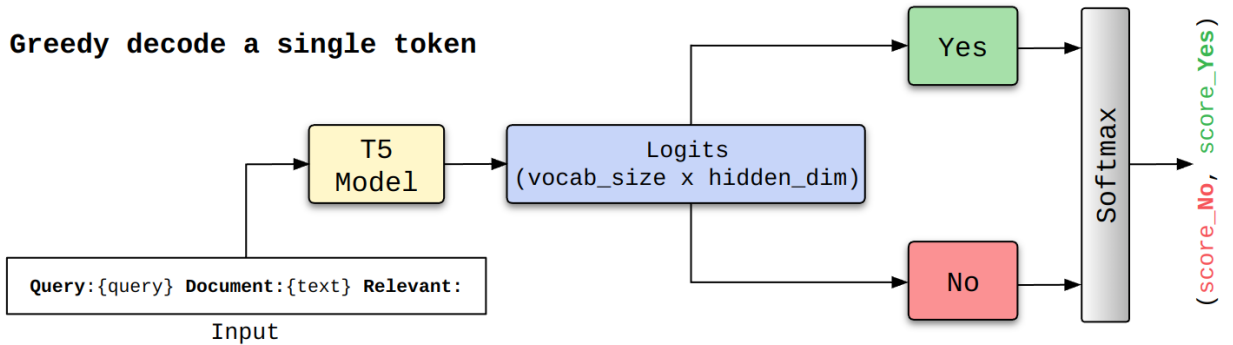


Figure 11: monoT5's inference process

In our experiments, Pyserini BM25 was employed as first-stage retrieval, retrieving 50 documents to be reranked in the second stage. In the second stage, we utilized the model unicamp-dl/mt5-base-en-pt-msmarco-v2, a sequence-to-sequence reranker trained on pairs of queries and documents in English and Portuguese from the dataset [33].

5.2 Retrievers Results

The results achieved with the various retrievers are presented in Table 6.

Table 6: Retriever comparison.

Metric	ADA-002	Custom ADA-002	Hybrid-BM25-ADA-002	Hybrid-BM25-Custom ADA-002	BM25	BM25 + Reranker
MRR@10	0.565	0.665	0.758	0.850	0.879	0.919
R@3	0.628	0.735	0.829	0.921	0.914	0.971
R@5	0.692	0.835	0.879	0.943	0.971	0.985
R@7	0.750	0.871	0.921	0.964	0.985	0.992
R@9	0.814	0.921	0.957	0.979	0.985	1

The multi-stage pipeline was able to achieve the best results in MRR@10 and Recall@k.

6 Conclusions

The implementation of RAG systems faces challenges such as the effective integration of retrieval models, efficient representation learning, the diversity of data, the optimization of computational efficiency, evaluation, and text generation quality. Faced with these constantly evolving obstacles, this article proposes best practices for the implementation, optimization, and evaluation of RAG on a Brazilian Portuguese dataset, focusing on a simplified pipeline for inference and experimentation.

So far, we have introduced the main components and methods along with their results and gaps. In this section, we will discuss key points that contribute to the performance improvement in RAG applications. We will start by discussing the relationship between the quality of the retriever and the achieved performance, in which our approach showed a significant improvement of MRR@10 by 35.4% compared to the baseline. Next, we will address the impact of input size on performance. In this domain, we observed that it is possible to enhance the best information retrieval strategy by 2.4% through input size optimization. Finally, we will present the complete architecture of RAG with our recommendations. When evaluating the final accuracy of our approach, we reached 98.61%, representing an improvement of 40.73 in less degradation score compared to the baseline.

6.1 Retriever Score versus Performance

As mentioned in section 5.1, the performance of information/chunk retrieval, measured by the MRR@10 metric, varies between (0.565, 0.919), as detailed in Table 6. This variation represents approximately 35.4%. It is important to highlight that the RAG’s performance is directly influenced by the quality of the retriever. Figure 12 shows the relationship between the retrieval metric MRR@10 metric and the degradation score for the studied retrieval methods.

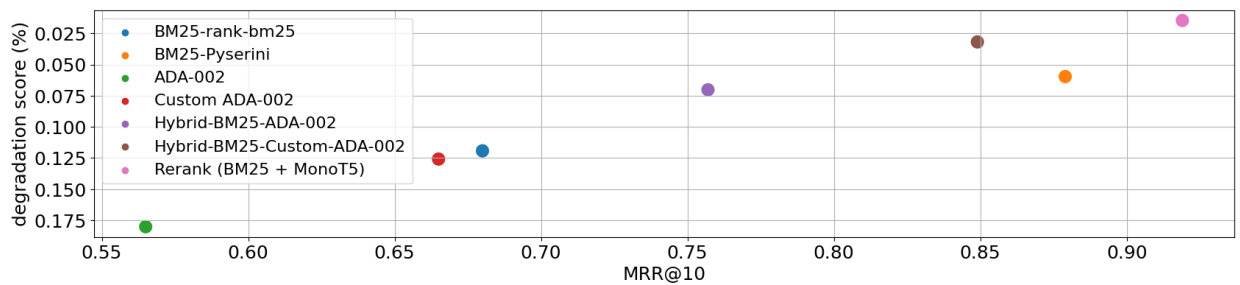


Figure 12: Retriever effectiveness vs RAG Performance. x axis is the MRR@10 metric and y axis is the degradation (where 0 is the perfect scenario) score.

6.2 Input Size versus Performance

We observed that the best performance was achieved with the retrieval of 3 chunks using the retrieve-and-rerank strategy, as shown in Table 7. The use of a reranker (Figure 9) showed improved information retrieval in our tests. With this same configuration, the Gemini Pro achieved performance similar to gpt-4, as indicated in Table 8.

Despite achieving perfect recall for 9 chunks, as evidenced in Table 6, using an input with 9000 tokens, 6000 more than the best scenario (3 chunks), did not result in the best performance. As discussed in Section 4.2, the quality of

Table 7: Performance of number of chunks retrieved with gpt-4.

# Retrieved Chunks	ADA-002	Custom ADA-002	BM25	Hybrid	BM25 + Reranker
3	6.19	6.41	7.10	7.31	7.44
5	6.29	6.61	7.32	7.37	7.43
7	6.42	6.82	7.17	7.20	7.32
9	6.57	6.88	7.22	7.34	7.37

Table 8: Performance of best retriever RAG.

Model	Retriever Method	# Retrieved Chunks	Degradation
gpt-4	ADA-002	9	-13.0%
gpt-4	ADA-002 Custom	9	-8.8%
gpt-4	BM25	5	-3%
gpt-4	Hybrid	5	-2.3%
gpt-4	BM25 + Reranker	3	-1.4%
Gemini Pro	BM25 + Reranker	3	-2.2%

RAG is directly related to the input size and the position where the answer is located. Therefore, the final results confirm the observations made in our experiments.

Moreover, from a cost perspective, it is crucial to avoid overloading the LLM with a large number of input tokens, as the cost is also based on the amount of input text.

It is important to note that the results obtained in this study cannot be considered as a generalization for other datasets. Exploratory Data Analysis and the use of good retriever practices, as presented in [34], are always a solid path to achieving good results.

6.3 Final Results

Despite this work being grounded in a single dataset, it is always crucial to emphasize the importance of data quality. In a simplified manner, as illustrated in Figure 13, data quality in RAG can be divided into Input, Retriever, and Evaluation.

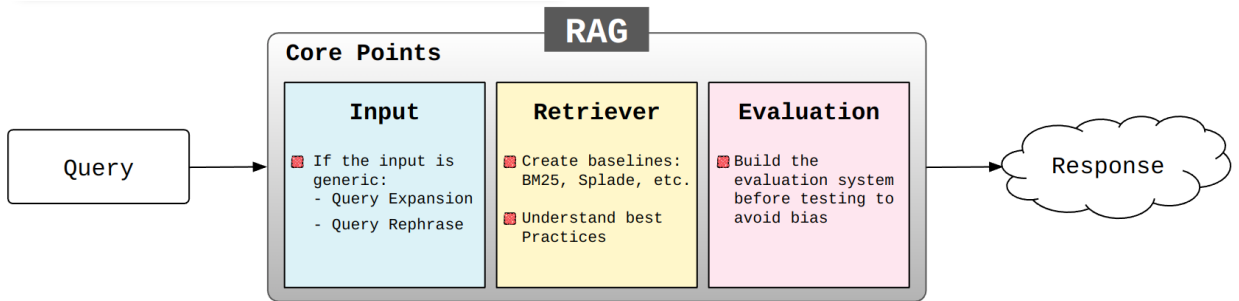


Figure 13: Core Points in RAG.

- **Input:** How are the queries formulated? Are they synthetic or generic? What is the application’s purpose? Building a RAG for a chatbot differs significantly from constructing a RAG for extracting information from a long and complex document.
- **Retriever:** How do the data behave in information retrieval? Are the queries strongly linked by keywords in the text? What is the cosine similarity of query-to-query, query-to-document, and document-to-document?
- **Evaluation:** How are the data measured? Defining metrics and success rates, as in the first experiment, is always a safe path to avoid biases. Build the evaluation system before testing, avoiding bias.

In conclusion, the main contribution of this work was to identify and present the best possible configuration of techniques and parameters for a RAG application. Figure 14 provides a summarized overview of the end-to-end experiment results for the discussed approaches, and the best practices recommended by this study achieve a final accuracy of 98.61%, representing an improvement of 40.73 percentage points compared to the baseline.

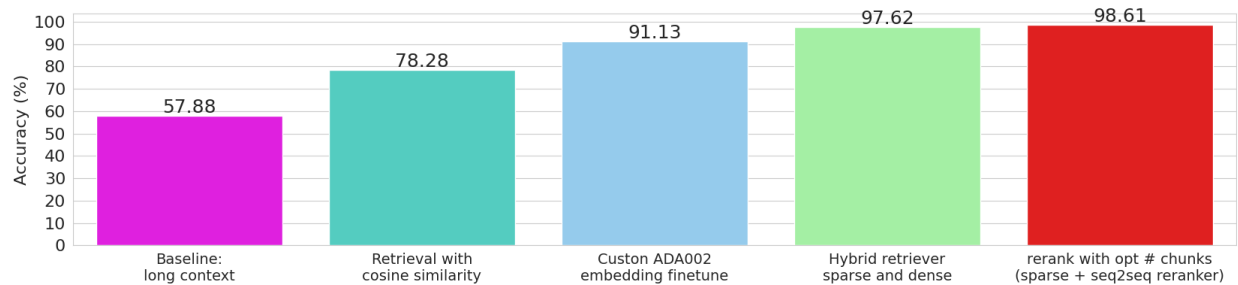


Figure 14: RAG Performance: Performance evolution on the Harry Potter dataset, where an accuracy of 100% is considered the relative maximum.

6.4 Future Work

We will expand our search to cover additional data sets, preferably those containing real data that already have questions and the reference documents for the answers. By using such datasets, we intend to explore techniques related to segmentation and chunk construction, as elaborated in Appendix B, Section B.

References

- [1] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, “Language models are few-shot learners,” 2020.
- [2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” 2021.
- [3] Y. Zhang, Y. Li, L. Cui, D. Cai, L. Liu, T. Fu, X. Huang, E. Zhao, Y. Zhang, Y. Chen, L. Wang, A. T. Luu, W. Bi, F. Shi, and S. Shi, “Siren’s song in the ai ocean: A survey on hallucination in large language models,” 2023.
- [4] J. Pereira, R. Fidalgo, R. Lotufo, and R. Nogueira, “Visconde: Multi-document qa with gpt-3 and neural reranking,” 2022.
- [5] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, and H. Wang, “Retrieval-augmented generation for large language models: A survey,” 2023.
- [6] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, “Squad: 100,000+ questions for machine comprehension of text,” 2016.
- [7] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, “Bleu: a method for automatic evaluation of machine translation,” in *Proceedings of the 40th annual meeting on association for computational linguistics*. Association for Computational Linguistics, 2002, pp. 311–318.
- [8] C.-Y. Lin, “Rouge: a package for automatic evaluation of summaries,” 2004.
- [9] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, and C. Zhu, “G-eval: Nlg evaluation using gpt-4 with better human alignment,” 2023.
- [10] E. Horvitz. (2023) The power of prompt. [Online]. Available: <https://www.microsoft.com/en-us/research/blog/the-power-of-prompting/>
- [11] A. Radford and K. Narasimhan, “Improving language understanding by generative pre-training,” 2018. [Online]. Available: <https://api.semanticscholar.org/CorpusID:49313245>
- [12] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, “Language models are unsupervised multitask learners,” 2019. [Online]. Available: <https://api.semanticscholar.org/CorpusID:160025533>
- [13] G. Alperovich, “The secret sauce behind 100k context window in llms: all tricks in one place,” <https://blog.gopenai.com/how-to-speed-up-llms-and-use-100k-context-window-all-tricks-in-one-place-ffd40577b4c>, 2023/May, accessed: 01/02/2024.

- [14] O. Press, N. A. Smith, and M. Lewis, “Train short, test long: Attention with linear biases enables input length extrapolation,” 2022.
- [15] S. Chen, S. Wong, L. Chen, and Y. Tian, “Extending context window of large language models via positional interpolation,” 2023.
- [16] P. Xu, W. Ping, X. Wu, L. McAfee, C. Zhu, Z. Liu, S. Subramanian, E. Bakhturina, M. Shoenybi, and B. Catanzaro, “Retrieval meets long context large language models,” 2023.
- [17] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, “Lost in the middle: How language models use long contexts,” 2023.
- [18] G. Kamradt, “Pressure testing claude-2.1 200k via needle-in-a-haystack,” 2023. [Online]. Available: https://github.com/gkamradt/LLMTest_NeedleInAHaystack
- [19] J. Liu, “Llamaindex,” 2022. [Online]. Available: https://github.com/jerryliu/llama_index
- [20] G. Mohandas and P. Moritz. (2023) Building rag-based llm applications for production. [Online]. Available: <https://www.anyscale.com/blog/a-comprehensive-guide-for-building-rag-based-llm-applications-part-1>
- [21] J. Lin, R. F. Nogueira, and A. Yates, “Pretrained transformers for text ranking: BERT and beyond,” *CoRR*, vol. abs/2010.06467, 2020. [Online]. Available: <https://arxiv.org/abs/2010.06467>
- [22] S. Humeau, K. Shuster, M. Lachaux, and J. Weston, “Real-time inference in multi-sentence tasks with deep pretrained transformers,” *CoRR*, vol. abs/1905.01969, 2019. [Online]. Available: <http://arxiv.org/abs/1905.01969>
- [23] G. M. Rosa, R. C. Rodrigues, R. Lotufo, and R. Nogueira, “Yes, bm25 is a strong baseline for legal case retrieval,” 2021.
- [24] D. Brown, “Rank-BM25: A Collection of BM25 Algorithms in Python,” 2020. [Online]. Available: <https://doi.org/10.5281/zenodo.4520057>
- [25] J. Lin, X. Ma, S.-C. Lin, J.-H. Yang, R. Pradeep, and R. Nogueira, “Pyserini: An easy-to-use python toolkit to support replicable ir research with sparse and dense representations,” 2021.
- [26] M. Henderson, R. Al-Rfou, B. Strope, Y. hsuan Sung, L. Lukacs, R. Guo, S. Kumar, B. Miklos, and R. Kurzweil, “Efficient natural language response suggestion for smart reply,” 2017.
- [27] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, “Reciprocal rank fusion outperforms condorcet and individual rank learning methods,” *Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval*, 2009. [Online]. Available: <https://api.semanticscholar.org/CorpusID:12408211>
- [28] R. F. Nogueira, W. Yang, K. Cho, and J. Lin, “Multi-stage document ranking with BERT,” *CoRR*, vol. abs/1910.14424, 2019. [Online]. Available: <http://arxiv.org/abs/1910.14424>
- [29] J. Guo, Y. Fan, L. Pang, L. Yang, Q. Ai, H. Zamani, C. Wu, W. B. Croft, and X. Cheng, “A deep look into neural ranking models for information retrieval,” 2019.
- [30] B. Mitra and N. Craswell, “An introduction to neural information retrieval,” *Foundations and Trends® in Information Retrieval*, vol. 13, no. 1, pp. 1–126, 2018. [Online]. Available: <http://dx.doi.org/10.1561/15000000061>
- [31] R. F. Nogueira, Z. Jiang, and J. Lin, “Document ranking with a pretrained sequence-to-sequence model,” *CoRR*, vol. abs/2003.06713, 2020. [Online]. Available: <https://arxiv.org/abs/2003.06713>
- [32] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, “Exploring the limits of transfer learning with a unified text-to-text transformer,” 2023.
- [33] L. H. Bonifacio, I. Campiotti, R. de Alencar Lotufo, and R. F. Nogueira, “mmarco: A multilingual version of MS MARCO passage ranking dataset,” *CoRR*, vol. abs/2108.13897, 2021. [Online]. Available: <https://arxiv.org/abs/2108.13897>
- [34] W. X. Zhao, J. Liu, R. Ren, and J.-R. Wen, “Dense text retrieval based on pretrained language models: A survey,” 2022.

A PROMPTS

All prompts were translated into English.

Prompt: Make QA dataset

```
You will receive a document that needs you to create 1 question. It is necessary that you
create only '1 question' for the document.
'I will use the created question to be answered given the entire Harry Potter book, so the
question needs to be clear and always detailed to have only one unique answer throughout
the book.'
## Follow exactly the following instructions, take the time necessary to produce a quality
question:
1. You need to create a question similar to the 'Squad' dataset, where the answer to this
dataset is a span of the text. The question must be unique, and the answer must be
contained within the document.
2. The answer to the question should be a faithful copy of the context of the document.
3. Create a detailed question where the answer must have more than 4 words.
4. Avoid generic questions.
The output should be a json with the characteristics of the example below:
Take your time, make sure the json does not have errors. Check and make sure there are no
errors when loading this json into a Python code using 'json.loads'.
{"question": "Write the question here.",
"answer": "Write the answer here. It must be a span and needs to be contained in the context
."},
# Begin of the document:
{document}
```

Prompt: Evaluator

```
You will receive two texts that need to be compared according to the scoring system
described below:

# Scoring system description.
- Score 1: The response has no relation to the reference.
- Score 3: The response has little relevance but is not aligned with the reference.
- Score 5: The response has moderate relevance but contains inaccuracies.
- Score 7: The response is aligned with the reference but has minor omissions.
- Score 10: The response is completely accurate and aligns perfectly with the reference.

## Start of text 1:
{text_1}
###
## Start of text 2:
{text_2}

Evaluate with great care and thoroughly, take the time necessary to provide a quality score.

# Expected scores
- Score 1: ["only the crazy know.", "everything is divine and wonderful."],
- Score 3: ["A car has 4 wheels.", "Is there any blue car with 5 wheels."],
- Score 5: ["Harry Potter is friends with Hermione and Snape.", "Hermione has several
friends, including 3 men."],
- Score 7: ["Harry Potter is friends with Hermione.", "Hermione is friends with some wizards
, including one who wears glasses."],
- Score 10: ["Brazil won 5 FIFA World Cup in soccer", "Brazil is a five-time world champion
."],

'Respond with only a numerical score.'
```

B Sentence Window

There are a wide variety of techniques available for dealing with RAG and new approaches are constantly emerging. In this work we highlight the main existing strategies, although we have also explored others, such as the RAG Sentence Window approach.

The sentence window chunk constitutes a strategy embedded in the llama-index¹, which employs the small-to-large approach. The entire chunk retrieval process is based on cosine similarity. Initially, a single sentence is retrieved, followed by expanding (window) around the retrieved sentence to construct a more extensive chunk. This method implies that the most relevant context is always centered and retrieved first, as opposed to being at the edges of a chunk boundary. Figure 15 illustrates the comparison between naive chunks and those obtained through the "sentence window" strategy.

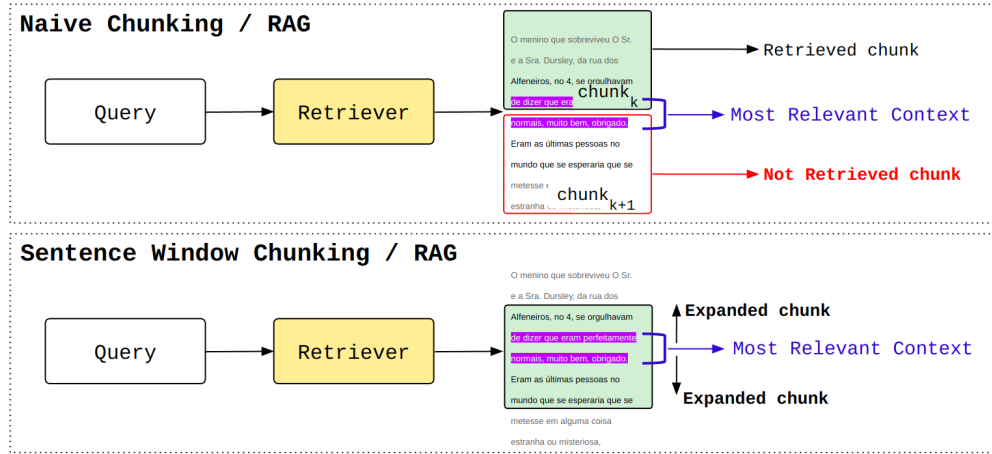


Figure 15: Comparison between the naive chunking and small to big strategies, where the latter initially retrieves a single sentence and then performs windowing with padding on both sides.

Despite being a promising technique, the results on our dataset were inferior to the naive strategy, as indicated in Table 9.

Table 9: Performance of the RAG Sentence Window.

Model	Average Score	Degradation
gpt-4	5.77	-23.5%
gpt-4-1106-preview	5.51	-24.7%
gpt-3.5-turbo-1106	5.60	-23.8%

Our hypothesis for the low performance includes:

1. The construction of the question and answer occurred after the "chunk" cut, so there is no answer between two chunks.
2. Embedding sentence by sentence in the Harry Potter book causes ambiguity: there are multiple sentences with the name *Harry Potter*, which adds a significant challenge to the step of retrieving the correct sentence.

¹https://docs.llamaindex.ai/en/stable/examples/node_postprocessor/MetadataReplacementDemo.html